Arquitectura de Computadoras — Proyecto 2

Diseño e implementación de un procesador RISC-V superescalar Dual-Issue con optimización microarquitectónica

1. Introducción y objetivos

Los procesadores modernos incorporan técnicas para aprovechar el paralelismo a nivel de instrucciones (Instruction-Level Parallelism, ILP). Una arquitectura superescalar permite emitir y ejecutar más de una instrucción por ciclo, siempre que sus dependencias y los recursos disponibles lo permitan.

El objetivo es diseñar e implementar un procesador RISC-V RV32I Dual-Issue In-Order a partir de un pipeline de cinco etapas. Los estudiantes deberán ampliar el datapath y el control para soportar emisión dinámica, asignación de recursos, detección de dependencias, forwarding y manejo de hazards. Adicionalmente, cada grupo propondrá e implementará una optimización microarquitectónica propia.

El resultado deberá permitir comparar tres configuraciones: (A) pipeline Single-Issue original, (B) procesador Dual-Issue y (C) procesador Dual-Issue con la optimización propuesta.

2. Arquitectura propuesta

Se conservarán las cinco etapas IF, ID, EX, MEM y WB. La emisión será en orden (In-Order Issue) y de hasta dos instrucciones por ciclo. No se requiere ejecución Out-of-Order, Tomasulo ni renombramiento de registros.

Se implementarán dos vías de ejecución capaces de aceptar las instrucciones RV32I soportadas por el procesador original. Ambas vías podrán ejecutar operaciones ALU y podrán recibir instrucciones de memoria o de salto; sin embargo, compartirán los recursos correspondientes. El procesador contará con dos ALU, un único puerto de memoria de datos y un único recurso de resolución de branches. Por tanto, podrá emitir hasta dos operaciones ALU, pero como máximo una operación de memoria y un branch por ciclo, siempre que se cumplan las restricciones de dependencia y control.

La Issue Unit deberá decidir dinámicamente si emite dos instrucciones, una o ninguna, y asignar los recursos disponibles sin alterar el orden del programa. No será necesario duplicar los puertos de memoria ni la unidad de resolución de saltos.

3. Requerimientos técnicos

3.1. Fetch y Decode

Obtener y decodificar hasta dos instrucciones consecutivas por ciclo. Ajustar la actualización del PC y conservar las instrucciones que no puedan emitirse inmediatamente, evitando pérdidas y ejecuciones duplicadas.

3.2. Datapath y Register File

Extender el datapath con dos vías de ejecución, dos ALU, los registros de pipeline necesarios, los multiplexores de asignación de recursos compartidos y un Register File con hasta cuatro lecturas y dos escrituras por ciclo. Resolver correctamente escrituras simultáneas y mantener la semántica del programa.

3.3. Issue Unit

Implementar una unidad que determine si puede emitirse un par de instrucciones según dependencias RAW, disponibilidad de unidades funcionales, conflictos estructurales, operaciones de memoria, saltos y hazards con instrucciones anteriores. La asignación de instrucciones a los recursos deberá permitir, por ejemplo, emitir una operación ALU junto con una operación Load/Store independientemente de cuál aparezca primero en el par, cuando no existan otras restricciones.

No se emitirán simultáneamente instrucciones con una dependencia RAW que no pueda resolverse dentro del mismo ciclo. Si solo puede emitirse la instrucción más antigua, la más joven deberá conservarse para el

ciclo siguiente, respetando el orden del programa.

3.4. Hazard Detection y Forwarding

Ampliar el forwarding entre ambas vías y las etapas del pipeline. Introducir stalls cuando no haya resultados disponibles y asegurar la prioridad correcta cuando varios productores escriban un mismo registro. El control debe conservar el orden de emisión y evitar instrucciones perdidas o repetidas.

3.5. Memoria

Soportar una instrucción Load/Store junto con una instrucción ALU independiente, en cualquier orden dentro del par. Se asumirá una memoria de datos de un solo puerto; no se requieren dos accesos de memoria simultáneos. Si ambas instrucciones requieren acceder a memoria, deberán emitirse en ciclos distintos.

3.6. Branches y control hazards

Soportar saltos condicionales e incondicionales en cualquiera de las dos posiciones del par de instrucciones. Se dispondrá de un único recurso de resolución de branches, por lo que no se requiere resolver dos saltos simultáneamente. La Issue Unit deberá detectar dependencias y restricciones de control antes de emitir un par.

Cuando un salto sea tomado, se anularán exclusivamente las instrucciones más jóvenes que correspondan al camino incorrecto, incluida la otra instrucción del par cuando el branch sea la primera y aquella sea más joven. Si el branch es la segunda instrucción, la primera, por ser más antigua, deberá conservarse. El PC deberá actualizarse correctamente.

4. Etapas de desarrollo

4.1. Primera etapa: pipeline y arquitectura inicial

Implementar y verificar el pipeline RV32I de cinco etapas, incluyendo forwarding, stalling y flushing. Iniciar la extensión superescalar con Dual Fetch, Dual Decode y las primeras modificaciones al datapath.

4.2. Segunda etapa: procesador superescalar funcional

Completar las dos vías, la Issue Unit, la asignación de recursos compartidos, el forwarding, los stalls y el manejo de memoria y branches. El procesador deberá ejecutar correctamente programas RV32I y emitir simultáneamente instrucciones independientes cuando sea posible.

4.3. Tercera etapa: optimización microarquitectónica

Cada grupo deberá identificar una limitación del procesador Dual-Issue, proponer una mejora adicional, implementarla y evaluar sus efectos. La propuesta deberá justificarse técnicamente y ser aprobada previamente. No se considerarán optimizaciones los requisitos ya obligatorios ni modificaciones exclusivamente al código Assembly.

Posibles alternativas de optimización:

- Predicción dinámica de saltos (por ejemplo, predictor de 2 bits).
- Branch Target Buffer (BTB) para anticipar direcciones de salto.
- Instruction Queue para desacoplar Fetch de la emisión.
- Mejoras en la política de la Issue Unit más allá de la asignación de recursos obligatoria.
- Mejoras adicionales de forwarding que no sustituyan el forwarding básico exigido.
- Caché de instrucciones o de datos, con una evaluación de su impacto.
- Mejoras adicionales en el acceso a memoria y la resolución de conflictos estructurales.

Se aceptarán otras propuestas justificadas. La optimización deberá conservar la corrección funcional y su efecto deberá medirse con métricas pertinentes; no es obligatorio que mejore el IPC en todos los programas.

5. Programas de prueba y evaluación experimental

5.1. Verificación funcional

Los grupos deberán elaborar pruebas de emisión simultánea de instrucciones independientes, dependencias RAW, forwarding entre vías, stalls, conflictos estructurales, branches en ambas posiciones del par (tomados y no tomados), asignación de recursos compartidos y escritura correcta de registros y memoria. Las tres arquitecturas deberán producir resultados funcionalmente equivalentes.

5.2. Programas de comparación

Cada grupo deberá desarrollar un (1) programa Assembly propio de evaluación. Adicionalmente, el profesor proporcionará un segundo programa de prueba, que también deberá ejecutarse y analizarse. Ambos programas deberán correr sin modificaciones sobre las tres arquitecturas, con iguales datos de entrada y condiciones comparables.

El programa propio deberá tener suficiente complejidad para evidenciar el aprovechamiento del paralelismo de instrucciones y permitir analizar la optimización seleccionada. No será suficiente utilizar únicamente una secuencia trivial de operaciones independientes. El programa proporcionado por el profesor servirá como prueba adicional de corrección y rendimiento.

5.3. Comparación obligatoria

Para cada uno de los dos programas, se compararán: A) pipeline Single-Issue, B) Dual-Issue sin optimización y C) Dual-Issue optimizado. Se reportarán instrucciones retiradas, ciclos de ejecución, IPC, CPI y speedup respecto a A y B. Se verificará que las tres configuraciones producen el mismo resultado.

IPC = instrucciones retiradas / ciclos; CPI = ciclos / instrucciones retiradas; Speedup(A a B) = ciclos(A) / ciclos(B).

Los ciclos deberán medirse con criterios idénticos de inicio y finalización. Cuando corresponda, se incluirán métricas adicionales, como stalls, utilización de doble emisión, precisión de predicción, tasa de aciertos de caché o recursos de hardware.

5.4. Análisis

El análisis deberá explicar la mejora del Dual-Issue frente al pipeline original, las causas que impiden alcanzar el IPC ideal de 2, la limitación que aborda la optimización, su impacto respecto al Dual-Issue sin optimizar y las condiciones en que resulta útil o no. Las conclusiones deberán sustentarse en mediciones.

6. Consideraciones finales

Se espera un diseño funcional, verificable y documentado que evidencie las dificultades del paralelismo a nivel de instrucciones y los compromisos entre complejidad microarquitectónica y rendimiento. Las reglas de entrega, fechas, puntajes y presentación se especificarán en un documento independiente.